
Corrigé Programmation

Découverte de la programmation, VS Code,
Python, C, HTML-CSS, PHP

Année universitaire 2025–2026

1 Découverte de la programmation et des outils de développement

Correction 1.1 – QCM : Généralités sur la programmation

-) **Réponses : ii., iv.** Un programme source est écrit dans un langage de programmation (ii). Le résultat de la compilation est un programme exécutable (iv), pas le programme source. Le programme source n'est pas en langage machine (i) ni exécutable directement (iii).
-) **Réponses : i., iii.** Le langage machine est composé d'instructions en binaire (i), spécifiques à chaque type de processeur (iii). Il n'est pas compréhensible par l'humain (ii) et se distingue de l'assembleur (iv) qui est une représentation symbolique.
-) **Réponses : i., iii.** Le compilateur traduit le code source en langage machine (i) et produit un fichier exécutable (iii). L'exécution ligne par ligne (ii) est le rôle de l'interpréteur. Les langages interprétés (iv) n'ont pas besoin de compilateur.
-) **Réponses : ii., iv.** L'interpréteur exécute les instructions une par une (ii). Python est un langage interprété (iv). La traduction complète avant exécution (i) est le rôle du compilateur. Les langages interprétés (iii) ne nécessitent pas d'étape de compilation.

Correction 1.2 – Langages compilés vs interprétés

Langage

Compilé heightPython

Mixte (compilé en bytecode + interprété par JVM) heightJavaScript

Interprété heightC++

Ni compilé ni interprété (langage de balisage/style) height

Type heightC

Interprété heightJava

Interprété heightPHP

Compilé heightHTML/CSS

Attention

HTML et CSS ne sont **pas** des langages de programmation : HTML est un langage de balisage, CSS un langage de feuilles de style. Ils ne contiennent pas de logique algorithmique.

Correction 1.3 – Vocabulaire de la programmation

Terme

2. Suite finie et non ambiguë d'opérations permettant de résoudre un problème. height**b**. Code source
4. Erreur dans un programme entraînant un dysfonctionnement. height**d**. IDE
3. Interface permettant à des logiciels de communiquer entre eux. height**f**. Framework

Correction 1.4 – Les étapes de la création d'un programme

L'ordre chronologique correct est :

1. **Analyse du problème et définition des besoins**
2. **Conception de l'algorithme**
3. **Écriture du code source**
4. **Compilation ou interprétation du code source**
5. **Mise au point et tests (débogage)**
6. **Maintenance et évolution**

Correction 1.5 – Types d'erreurs

-) **Erreur de syntaxe** – Oubli d'un point-virgule.
-) **Erreur d'exécution** – Division par zéro.
-) **Erreur logique** – Résultat incorrect sans plantage.
-) **Erreur de syntaxe** – Variable non déclarée détectée à la compilation.
-) **Erreur d'exécution** – Accès hors limites d'un tableau.
-) **Erreur logique** – Le programme fonctionne mais ne fait pas ce qu'on attend.

Astuce

Les erreurs de syntaxe sont détectées **avant** l'exécution (à la compilation). Les erreurs d'exécution provoquent un **plantage**. Les erreurs logiques sont les plus difficiles à détecter car le programme s'exécute sans erreur apparente.

2 Présentation de VS Code

Correction 2.1 – Installation de VS Code

L'installation se fait en suivant les étapes de l'énoncé. Vérifiez que VS Code se lance correctement après l'installation.

Conseil

Cochez l'option « Add to PATH » lors de l'installation sous Windows pour pouvoir lancer VS Code depuis le terminal avec la commande `code`.

Correction 2.2 – Identification des éléments de l'interface

-) **Barre de menus** – En haut de la fenêtre (Fichier, Édition, Affichage, etc.).
-) **Éditeur / Zone d'édition** – Zone centrale où l'on saisit le code.
-) **Explorateur de fichiers** – Panneau latéral gauche affichant l'arborescence.
-) **Barre d'état** – En bas : langage, ligne/colonne, encodage.
-) **Panneau terminal / sortie** – En bas : Terminal, Problèmes, Sortie, Débogage.
-) **Barre d'activité** – Icônes à gauche : Explorateur, Recherche, Git, Extensions, etc.

Correction 2.3 – Raccourcis clavier essentiels

Action	Raccourci (Windows/Linux)
Ctrl+N Ouvrir un fichier	Ctrl+N Nouveau fichier
Ctrl+S Sauvegarder tout	Ctrl+S Sauvegarder
Ctrl+Z Refaire	Ctrl+Z Annuler
Ctrl+F Remplacer dans le fichier	Ctrl+F Rechercher dans le fichier
Ctrl+` Commenter/Décommenter	Ctrl+` Ouvrir le terminal
	Ctrl+` height

Correction 2.4 – Extensions VS Code

Langage	Nom de l'extension	Fonction principale
Python (Microsoft)	Auto-complétion, linting, débogage, exécution	Python
IntelliSense, débogage, compilation	C / C++	C/C++ (Microsoft)
HTML-CSS	Live Server (Ritwick Dey)	Rechargement en direct des pages web
PHP	Intelephense	PHP
	Auto-complétion, analyse statique, navigation	

Autres extensions utiles

Prettier (formatage), GitLens (Git avancé), Auto Rename Tag (HTML), Bracket Pair Colorizer (parenthèses colorées).

Correction 2.5 – Le terminal intégré

1. Le terminal intégré s'ouvre avec **Ctrl+`**.
2. Explication des commandes :
 - `pwd` : affiche le répertoire de travail courant.
 - `ls` : liste les fichiers et dossiers du répertoire courant.
 - `cd ..` : remonte au répertoire parent.
 - `cd ~` : va au répertoire personnel de l'utilisateur.
 - `mkdir test_vscode` : crée un dossier `test_vscode`.
 - `cd test_vscode` : entre dans ce dossier.
 - `touch fichier_test.txt` : crée un fichier vide.
 - `ls` : vérifie la présence du fichier créé.
3. Pour supprimer le dossier : `rm -r test_vscode`

3 Configuration de l'environnement de développement

Correction 3.1 – Vérification des outils installés

Les résultats dépendent de la machine. Voici un exemple de sortie attendue :

```
$ python3 --version
Python 3.12.4
$ gcc --version
gcc (Ubuntu 13.2.0) ...
```

```
$ php --version
PHP 8.3.6 ...
$ node --version
v20.14.0
$ git --version
git version 2.43.0
```

Si un outil n'est pas installé, le terminal affiche « command not found ».

Correction 3.2 – Installation de Python

Suivre les étapes de l'énoncé. Après installation, la vérification donne :

```
$ python3 --version
Python 3.12.4
$ python3 test.py
Hello, Python !
```

Important

Sous Windows, si vous avez oublié de cocher « Add Python to PATH », vous pouvez :

1. Relancer l'installateur et choisir « Modify ».
2. Ajouter manuellement le chemin dans les variables d'environnement.

Correction 3.3 – Installation du compilateur GCC

Après installation :

```
$ gcc --version
gcc (Ubuntu 13.2.0) ...
$ gcc test.c -o test
$ ./test
Hello, C !
```

Sous Windows

Si vous utilisez MinGW-w64, la commande de compilation est : `gcc test.c -o test.exe` puis `..exe` pour exécuter.

Correction 3.4 – Installation de PHP

Après installation :

```
$ php --version
PHP 8.3.6 ...
$ php test.php
Hello, PHP !
```

Correction 3.5 – Structure d'un projet

Voici un exemple de structure cohérente :

```
mon_projet/
|-- python/
|   |-- scripts/
|   |-- tests/
```

```
|-- c/
|   |-- src/
|   |-- include/
|   |-- build/
|-- web/
|   |-- html/
|       |-- index.html
|       |-- pages/
|   |-- css/
|       |-- style.css
|   |-- php/
|       |-- index.php
|       |-- includes/
|-- assets/
|   |-- images/
|   |-- fonts/
|-- docs/
|   |-- README.md
|-- .gitignore
```

Bonnes pratiques

Séparez toujours les fichiers sources des fichiers compilés (build/), les ressources (assets/) et la documentation (docs/). Utilisez un fichier `.gitignore` pour exclure les fichiers inutiles du versionnage.

Correction 3.6 – Fichier settings.json

```
{
  "editor.tabSize": 4,
  "editor.formatOnSave": true,
  "python.defaultInterpreterPath": "/usr/bin/python3",
  "html.format.enable": true,
  "css.format.enable": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "[html]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode"
  },
  "[css]": {
    "editor.defaultFormatter": "esbenp.prettier-vscode"
  }
}
```

4 Initiation et apprentissage de Python

Correction 4.1 – Premiers pas : print et input

```
nom = input("Entrez votre nom : ")
age = input("Entrez votre âge : ")
print(f"Bonjour {nom}, vous avez {age} ans.")
```

F-strings

Les **f-strings** (chaînes formatées, Python 3.6+) permettent d'insérer facilement des variables dans une chaîne en les plaçant entre accolades et en préfixant la chaîne avec **f**.

Correction 4.2 – Variables et types

Variable	Type
<code>heighta</code>	<code>int</code>
<code>heightb</code>	<code>float</code>
<code>heightc</code>	<code>float</code>
<code>heightd</code>	<code>bool</code>
<code>heighte</code>	<code>bool</code>
<code>heightf</code>	<code>int</code>
<code>heightg</code>	<code>int</code>
<code>heighth</code>	<code>str</code>
<code>heighti</code>	<code>str</code>

Typage dynamique

En Python, le type est déterminé automatiquement à l'exécution. `a + b` donne un `float` car l'addition d'un `int` et d'un `float` produit un `float`.

Correction 4.3 – Opérations et expressions

```
a = int(input("Entrez a : "))
b = int(input("Entrez b : "))

print(f"Somme : {a + b}")
print(f"Différence : {a - b}")
print(f"Produit : {a * b}")
print(f"Quotient entier : {a // b}")
print(f"Reste : {a % b}")
print(f"Puissance : {a ** b}")

print(f"Type somme : {type(a + b)}")
print(f"Type quotient : {type(a // b)}")
print(f"Type puissance : {type(a ** b)}")
```

Opérateurs Python

`//` est la division entière, `%` le modulo, `**` la puissance.

Correction 4.4 – Conditions

```
n = int(input("Entrez un entier : "))

# Signe
if n > 0:
    print("Positif")
elif n < 0:
    print("Négatif")
else:
    print("Zéro")

# Parité
if n % 2 == 0:
    print("Pair")
else:
    print("Impair")
```



```
# Multiple de 3
if n % 3 == 0:
    print("Multiple de 3")
else:
    print("Non multiple de 3")
```

Correction 4.5 – Boucle for

```
# 1. Nombres de 1 à 10
for i in range(1, 11):
    print(i)

# 2. Nombres pairs de 2 à 20
for i in range(2, 21, 2):
    print(i)

# 3. Somme de 1 à n
n = int(input("Entrez n : "))
somme = 0
for i in range(1, n + 1):
    somme += i
print(f"Somme = {somme}")

# 4. Table de multiplication
n = int(input("Entrez n : "))
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

`range(start, stop, step)`

`range(2, 21, 2)` génère 2, 4, 6, ..., 20. Attention : stop est exclu.

Correction 4.6 – Boucle while

```
# 1. Mot de passe
mdp = ""
while mdp != "python123":
    mdp = input("Entrez le mot de passe : ")
print("Accès autorisé !")

# 2. Puissances de 2
p = 1
while p <= 1000:
    print(p)
    p *= 2

# 3. Devine un nombre
import random
secret = random.randint(1, 100)
essai = 0
while essai != secret:
    essai = int(input("Devinez le nombre (1-100) : "))
    if essai > secret:
        print("Trop grand")
    elif essai < secret:
        print("Trop petit")
```

```
else:
    print("Gagné !")
```

Correction 4.7 – Listes

```
# 1. Liste de 1 à 10
liste = list(range(1, 11))
print(liste)

# 2. Premier et dernier
print(f"Premier : {liste[0]}")
print(f"Dernier : {liste[-1]}")

# 3. Ajouter 11
liste.append(11)
print(liste)

# 4. Supprimer 5
liste.remove(5)
print(liste)

# 5. Tri décroissant
liste.sort(reverse=True)
print(liste)

# 6. Somme et moyenne
somme = sum(liste)
moyenne = somme / len(liste)
print(f"Somme = {somme}, Moyenne = {moyenne}")
```

Index négatif

En Python, `liste[-1]` désigne le dernier élément, `liste[-2]` l'avant-dernier, etc.

Correction 4.8 – Fonctions

```
def est_pair(n):
    return n % 2 == 0

def factorielle(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def est_palindrome(mot):
    return mot == mot[::-1]

def moyenne(liste):
    return sum(liste) / len(liste)

# Tests
print(est_pair(4))    # True
print(factorielle(5))  # 120
print(est_palindrome("kayak"))  # True
print(moyenne([12, 15, 18]))  # 15.0
```

Palindrome

`mot[::-1]` est un *slice* qui inverse la chaîne. C'est une idiom Python très courant.

Correction 4.9 – Chaînes de caractères

```
phrase = input("Entrez une phrase : ")

print(f"Nombre de caractères : {len(phrase)}")
print(f"Majuscules : {phrase.upper()}")
print(f"Minuscules : {phrase.lower()}")

voyelles = "aeiouyAEIOUY"
nb_voyelles = sum(1 for c in phrase if c in voyelles)
print(f"Nombre de voyelles : {nb_voyelles}")

print(f"Inversée : {phrase[::-1]}")
```

5 Initiation et apprentissage du langage C**Correction 5.1 – Structure d'un programme C**

1. Les trois parties principales :
 - **Directive de préprocesseur** : `#include <stdio.h>`
 - **Fonction principale** : `int main()`
 - **Corps de la fonction** : entre `{ }`
2. Rôle de chaque ligne :
 - `#include <stdio.h>` : inclusion de la bibliothèque standard d'entrée/sortie.
 - `int main()` : point d'entrée du programme, retourne un entier.
 - `printf(...)` : affichage à l'écran.
 - `return 0` : indique que le programme s'est terminé normalement.
3. Sans `#include <stdio.h>`, le compilateur ne connaît pas `printf` et génère une erreur ou un avertissement.

Correction 5.2 – Variables et types en C**Description**

`int n = 42;`
Un entier valant 42

`float x = 3.14f;`
Un flottant valant 3.14

`char c = 'A';`
Caractère 'A'

`long l = 1000000L;`
Entier long 1000000

`double d = 2.718;`
Double précision 2.718

`char str[] = "Bonjour";`
Chaîne « Bonjour »

Déclaration C

`height` Un entier valant 42

`float x = 3.14f;` `height` Caractère 'A'

`long l = 1000000L;` `height` Double précision 2.718

`char str[] = "Bonjour";` `height`

Suffixes

En C, `f` suffixe un littéral `float`, `L` un `long`, et un littéral décimal sans suffixe est un `double`.

Correction 5.3 – printf et scanf

```
#include <stdio.h>

int main() {
    int n;
    float x;

    printf("Entrez un entier : ");
    scanf("%d", &n);
    printf("Vous avez saisi : %d\n", n);

    printf("Entrez un flottant : ");
    scanf("%f", &x);
    printf("Valeur avec 2 décimales : %.2f\n", x);

    return 0;
}
```

Format scanf

N'oubliez pas le & devant la variable dans `scanf` : on passe l'**adresse** de la variable pour que `scanf` puisse la modifier.

Correction 5.4 – Opérateurs en C

Variable	Valeur
3 (17 / 5 = 3, division entière) heightd	2 (17 modulo 5 = 2) heightc
3.4 (cast float, division réelle) heightf	17 (post-incrément : retourne 17, puis a passe à 18)
6 (pré-incrément : b passe à 6, puis retourne 6) heightb	1 (a=18 > 10, condition vraie) height

Post vs Pré-incrément

`a++` retourne la valeur **puis** incrémente. `++b` incrémente **puis** retourne. Après ces deux lignes, `a = 18` et `b = 6`.

Correction 5.5 – Conditions en C : maximum de trois nombres

```
#include <stdio.h>

int main() {
    int a, b, c, max;

    printf("Entrez trois nombres : ");
    scanf("%d %d %d", &a, &b, &c);

    max = a;
    if (b > max) max = b;
    if (c > max) max = c;

    printf("Le plus grand est : %d\n", max);
    return 0;
}
```

Correction 5.6 – Boucles en C

```
#include <stdio.h>

int main() {
    // 1. Nombres de 1 à 10
    for (int i = 1; i <= 10; i++) {
        printf("%d ", i);
    }
    printf("\n");

    // 2. Somme de 1 à n
    int n, somme = 0;
    printf("Entrez n : ");
    scanf("%d", &n);
    int i = 1;
    while (i <= n) {
        somme += i;
        i++;
    }
    printf("Somme = %d\n", somme);

    // 3. Motif d'étoiles
    int lignes = 5;
    for (int i = 1; i <= lignes; i++) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

Correction 5.7 – Tableaux en C

```
#include <stdio.h>

int main() {
    int tab[5];
    int somme = 0, min, max;

    // Saisie
    for (int i = 0; i < 5; i++) {
        printf("tab[%d] = ", i);
        scanf("%d", &tab[i]);
    }

    // Affichage
    printf("Contenu : ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", tab[i]);
        somme += tab[i];
    }
    printf("\n");

    // Somme et moyenne
    printf("Somme = %d\n", somme);
}
```

```
printf("Moyenne = %.2f\n", (float)somme / 5);

// Min et max
min = max = tab[0];
for (int i = 1; i < 5; i++) {
    if (tab[i] < min) min = tab[i];
    if (tab[i] > max) max = tab[i];
}
printf("Min = %d, Max = %d\n", min, max);

return 0;
}
```

Correction 5.8 – Fonctions en C

```
#include <stdio.h>

int carre(int n) {
    return n * n;
}

int est_pair(int n) {
    return n % 2 == 0 ? 1 : 0;
}

int factorielle(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    printf("Carre de 5 = %d\n", carre(5));
    printf("6 est pair ? %s\n", est_pair(6) ? "Oui" : "Non");
    printf("5! = %d\n", factorielle(5));

    int x = 10, y = 20;
    swap(&x, &y);
    printf("Après swap : x=%d, y=%d\n", x, y);

    return 0;
}
```

Pointeurs

La fonction `swap` utilise des **pointeurs** pour modifier les valeurs des variables appelantes. Sans pointeurs, les modifications seraient locales à la fonction.

6 Initiation et apprentissage de HTML-CSS

Correction 6.1 – Structure d'un document HTML

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Ma premiere page</title>
</head>
<body>
  <h1>Bienvenue !</h1>
  <p>Ceci est mon premier paragraphe en HTML.</p>
</body>
</html>
```

Correction 6.2 – Balises HTML courantes

Balise	Rôle
Paragraphe de texte height<a>	<p> Lien hypertexte (attribut href)
Image (attributs src, alt) height	 Liste non ordonnée (à puces)
Liste ordonnée (numérotée) height	 Élément d'une liste (dans ou)
Conteneur bloc générique (groupement) height<div>	<div> Conteneur en- ligne générique
Tableau de données height<table>	<table>

Correction 6.3 – Listes et liens

```
<!-- 1. Liste non ordonnee -->
<ul>
  <li>Python</li>
  <li>C</li>
  <li>Java</li>
  <li>PHP</li>
</ul>

<!-- 2. Liste ordonnee -->
<ol>
  <li>Ouvrir un editeur de texte</li>
  <li>Ecrire le code HTML</li>
  <li>Sauvegarder le fichier</li>
  <li>Ouvrir dans un navigateur</li>
</ol>

<!-- 3. Lien -->
<a href="https://code.visualstudio.com/">
  Telecharger VS Code
```

```

</a>

<!-- 4. Image -->


```

Correction 6.4 – Tableaux HTML

```

<table>
  <tr>
    <td colspan="3">Langages de programmation</td>
  </tr>
  <tr>
    <th>Langage</th>
    <th>Type</th>
    <th>Annee de creation</th>
  </tr>
  <tr>
    <td>C</td>
    <td>Compile</td>
    <td>1972</td>
  </tr>
  <tr>
    <td>Python</td>
    <td>Interprete</td>
    <td>1991</td>
  </tr>
  <tr>
    <td>Java</td>
    <td>Mixte</td>
    <td>1995</td>
  </tr>
  <tr>
    <td>PHP</td>
    <td>Interprete</td>
    <td>1995</td>
  </tr>
</table>

```

colspan / rowspan

colspan fusionne horizontalement, rowspan fusionne verticalement. Ces attributs sont utilisés sur les balises <td> ou <th>.

Correction 6.5 – Formulaires HTML

```

<form method="POST" action="traitement.php">
  <label for="nom">Nom :</label>
  <input type="text" id="nom" name="nom"><br>

  <label for="email">Email :</label>
  <input type="email" id="email" name="email"><br>

  <label for="mdp">Mot de passe :</label>
  <input type="password" id="mdp" name="mdp"><br>

  <label for="langage">Langage prefere :</label>
  <select id="langage" name="langage">

```



```

    <option value="c">C</option>
    <option value="python">Python</option>
    <option value="html">HTML-CSS</option>
    <option value="php">PHP</option>
</select><br>

<label>Niveau :</label>
<input type="radio" name="niveau" value="debutant"> Debutant
<input type="radio" name="niveau" value="intermediaire"> Intermédiaire
<input type="radio" name="niveau" value="avance"> Avancé<br>

<input type="checkbox" name="accepte" value="oui">
J'accepte les conditions<br>

<input type="submit" value="Envoyer">
</form>

```

Correction 6.6 – Premiers pas en CSS

```

/* 1. Paragraphes en bleu */
p { color: blue; }

/* 2. Centrer le h1 */
h1 { text-align: center; }

/* 3. Fond gris clair sur les div */
div { background-color: #f0f0f0; }

/* 4. Bordure sur les tableaux */
table { border: 2px solid black; }

/* 5. Police Arial sur toute la page */
body { font-family: Arial, sans-serif; }

```

Correction 6.7 – Sélecteurs CSS

Règle	Type de sélecteur	Éléments affectés
Sélecteur d'élément	Tous les <u><h1></u> Règle 2	Sélecteur de classe Règle 1
Tous les éléments avec <code>class="important"</code> height	Sélecteur d'identifiant Règle 3	L'unique élément avec <code>id="principal"</code> Règle 4
Sélecteur descendant	Les <u></u> à l'intérieur d'un <u><p></u> Règle 5	Sélecteur enfant
Les <u><p></u> enfants directs d'un <u><div></u> height	Pseudo-classe Règle 6	Les liens <u><a></u> au survol de la souris

Descendant vs Enfant

`p` em sélectionne tout `<em>` dans un `<p>` (même profond). `div > p` ne sélectionne que les `<p>` **directement** dans un `<div>`.

Correction 6.8 – Modèle de boîte CSS

1. Les quatre propriétés :

- **content** : la zone de contenu (texte, image...).
- **padding** : l'espace entre le contenu et la bordure.
- **border** : la bordure autour du padding.
- **margin** : l'espace entre la bordure et les éléments voisins.

2. Largeur totale :

$$\begin{aligned} \text{Largeur totale} &= \text{width} + \text{padding gauche} + \text{padding droit} + \text{border gauche} + \text{border droit} + \text{margin gauche} + \text{margin droit} \\ &= 200 + 10 + 10 + 5 + 5 + 15 + 15 = \mathbf{260 \text{ px}} \end{aligned}$$

box-sizing

Avec `box-sizing: border-box`, `width` inclut le padding et la bordure. La largeur totale deviendrait `width + margin = 200 + 15 + 15 = 230 px`.

Correction 6.9 – Mise en page avec Flexbox

```
<!-- HTML -->
<header>Mon Site</header>
<nav>
  <a href="#">Accueil</a>
  <a href="#">Cours</a>
  <a href="#">Exercices</a>
  <a href="#">Contact</a>
</nav>
<main>
  <article>Article 1</article>
  <article>Article 2</article>
  <article>Article 3</article>
</main>
<footer>Copyright 2025</footer>
```

```
/* CSS */
header {
  background: #005394;
  color: white;
  padding: 20px;
  text-align: center;
}

nav {
  display: flex;
  gap: 15px;
  background: #eee;
  padding: 10px;
}
```

```

main {
    display: flex;
    gap: 20px;
    padding: 20px;
}

article {
    flex: 1;
    background: #f5f5f5;
    padding: 15px;
    border: 1px solid #ddd;
}

footer {
    background: #333;
    color: white;
    text-align: center;
    padding: 10px;
}

```

flex: 1

flex: 1 fait que chaque article prend une part égale de l'espace disponible.

7 Initiation et apprentissage de PHP

Correction 7.1 – Premiers pas en PHP

```

<?php
echo "Bonjour, ceci est mon premier script PHP !";
echo "<br>";
echo "Date actuelle : " . date('d/m/Y');
echo "<br>";
phpinfo();
?>

```

Exécution en CLI

En ligne de commande : `php script.php`. La balise `<br>` ne sera interprétée que dans un navigateur. En CLI, utilisez `\n` à la place.

Correction 7.2 – Variables et types en PHP

Variable	Type PHP
<code>string height\$age</code>	<code>int height\$moyenne</code>
<code>float height\$actif</code>	<code>bool height\$notes</code>
<code>array height\$rien</code>	<code>NULL height</code>

Typage PHP

PHP est un langage à **typage dynamique** : le type est déterminé à l'exécution. Contrairement au C, on n'a pas besoin de déclarer le type d'une variable.

Correction 7.3 – Opérateurs en PHP

1. $\$a + \$b \rightarrow 13$
2. $\$a / \$b \rightarrow 3.3333...$ (division réelle en PHP)
3. $\$a \% \$b \rightarrow 1$
4. $\$a ** \$b \rightarrow 1000$
5. $\$a == 10 ? \text{«oui»} : \text{«non»} \rightarrow \text{oui}$ (comparaison lâche)
6. $\$a === \text{«10»} ? \text{«oui»} : \text{«non»} \rightarrow \text{non}$ (comparaison stricte : type différent)
7. $\$a != \$b ? \text{«diff»} : \text{«egal»} \rightarrow \text{diff}$
8. $\text{isset}(\$c) ? \text{«def»} : \text{«undef»} \rightarrow \text{undef}$ ($\$c$ n'est pas définie)

== vs ===

En PHP, `==` compare les valeurs après conversion de type, `===` compare valeurs **et** types. Toujours préférer `===` pour éviter les pièges.

Correction 7.4 – Conditions en PHP

```
<?php
$note = 15; // Exemple

if ($note >= 18) {
    echo "Excellent";
} elseif ($note >= 16) {
    echo "Tres bien";
} elseif ($note >= 14) {
    echo "Bien";
} elseif ($note >= 12) {
    echo "Assez bien";
} elseif ($note >= 10) {
    echo "Passable";
} else {
    echo "Insuffisant";
}
?>
```

Correction 7.5 – Boucles en PHP

```
<?php
// 1. Nombres de 1 a 10
for ($i = 1; $i <= 10; $i++) {
    echo $i . " ";
}
echo "\n";

// 2. Somme de 1 a n
$n = 100; // Exemple
$somme = 0;
$i = 1;
while ($i <= $n) {
    $somme += $i;
    $i++;
}
```

```

}
echo "Somme = $somme\n";

// 3. Foreach sur un tableau
$prenoms = ["Alice", "Bob", "Charlie"];
foreach ($prenoms as $prenom) {
    echo "Bienvenue $prenom !\n";
}

// 4. Table de multiplication
$n = 7; // Exemple
for ($i = 1; $i <= 10; $i++) {
    echo "$n x $i = " . ($n * $i) . "\n";
}
?>

```

Correction 7.6 – Tableaux en PHP

```

<?php
// 1. Tableau associatif
$eleves = [
    "Alice" => 16,
    "Bob" => 12,
    "Charlie" => 18,
    "Diana" => 14,
    "Eve" => 15
];

// 2. Afficher tous les élèves
foreach ($eleves as $nom => $note) {
    echo "$nom : $note\n";
}

// 3. Moyenne générale
$moyenne = array_sum($eleves) / count($eleves);
echo "Moyenne générale : $moyenne\n";

// 4. Meilleur élève
$meilleur = array_search(max($eleves), $eleves);
echo "Meilleur : $meilleur\n";

// 5. Tri par notes décroissantes
arsort($eleves);
print_r($eleves);
?>

```

Fonctions de tableau

`arsort()` trie un tableau associatif par valeurs décroissantes en préservant les clés. `asort()` pour l'ordre croissant.

Correction 7.7 – Fonctions en PHP

```

<?php
function est_pair($n) {
    return $n % 2 === 0;
}

```

```

function factorielle($n) {
    $result = 1;
    for ($i = 2; $i <= $n; $i++) {
        $result *= $i;
    }
    return $result;
}

function est_palindrome($mot) {
    return $mot === strrev($mot);
}

function valider_email($email) {
    return filter_var($email, FILTER_VALIDATE_EMAIL) !== false;
}

// Tests
echo est_pair(4) ? "Pair" : "Impair"; // Pair
echo factorielle(5); // 120
echo est_palindrome("kayak") ? "Oui" : "Non"; // Oui
echo valider_email("test@mail.com") ? "Valide" : "Invalide"; // Valide
?>

```

filter_var

PHP offre `filter_var()` avec des filtres prédéfinis comme `FILTER_VALIDATE_EMAIL`, `FILTER_VALIDATE_URL`, etc. Toujours l'utiliser plutôt que des expressions régulières maison.

Correction 7.8 – Formulaires et PHP

Formulaire HTML :

```

<form method="POST" action="traitement.php">
    <label>Nom :</label>
    <input type="text" name="nom" required><br>

    <label>Email :</label>
    <input type="email" name="email" required><br>

    <label>Message :</label>
    <textarea name="message" required></textarea><br>

    <input type="submit" value="Envoyer">
</form>

```

Script de traitement traitement.php :

```

<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nom = htmlspecialchars(trim($_POST["nom"]));
    $email = htmlspecialchars(trim($_POST["email"]));
    $message = htmlspecialchars(trim($_POST["message"]));

    // Vérification champs remplis
    if (empty($nom) || empty($email) || empty($message)) {
        echo "Tous les champs sont obligatoires.";
        exit;
    }
}

```

```
// Validation email
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Email invalide.";
    exit;
}

// Affichage sécurisé
echo "Nom : $nom<br>";
echo "Email : $email<br>";
echo "Message : $message<br>";
}
?>
```

Sécurité

htmlspecialchars() convertit les caractères spéciaux en entités HTML, empêchant les attaques XSS. trim() supprime les espaces en début et fin de chaîne.

Correction 7.9 – Sessions PHP

Page de connexion login.php :

```
<?php
session_start();

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $login = $_POST["login"];
    $mdp = $_POST["motdepasse"];

    if ($login === "admin" && $mdp === "admin123") {
        $_SESSION["login"] = $login;
        header("Location: protegee.php");
        exit;
    } else {
        echo "Identifiants incorrects.";
    }
}
?>

<form method="POST">
    Login : <input type="text" name="login"><br>
    Mot de passe : <input type="password" name="motdepasse"><br>
    <input type="submit" value="Connexion">
</form>
```

Page protégée protegee.php :

```
<?php
session_start();

if (!isset($_SESSION["login"])) {
    header("Location: login.php");
    exit;
}

echo "Bienvenue " . htmlspecialchars($_SESSION["login"]);
echo " <a href='deconnexion.php'>Deconnexion</a>";
?>
```

Page de déconnexion `deconnexion.php` :

```
<?php
session_start();
session_destroy();
header("Location: login.php");
exit;
?>
```

Sécurité des sessions

En production, ne stockez jamais de mots de passe en dur! Utilisez un hachage (`password_hash` / `password_verify`) et une base de données.